TermXT Scripting Language Manual

Reference and implementation guide for the TermXT interpreter in xTerminal

Scope

This manual documents the language as implemented by the provided TermXT C# class. Where the help text and the implementation differ, this guide calls that out explicitly.

Document type	Language manual / reference
Platform target	Windows ([SupportedOSPlatform("Windows")])
Primary file command	xt
Documented version	The TermXT implementation supplied in the conversation

Contents

1. Introduction and execution model
2. Invoking the xt command
3. Script structure, comments, whitespace, and interpolation
4. Variables, assignment, and right-hand-side transforms
5. Input, output, command execution, and pipelines
6. Conditions and branching
7. Loops and iteration
8. Functions and return values
9. Error handling and status variables
10. File operations and timing
11. Built-in variables and runtime state
12. Script checker behavior
13. Cookbook examples
14. Implementation quirks and caveats
15. Quick reference

> **Note.** The document uses ordinary section headings rather than an auto-generated Word TOC so that the file stays deterministic and easy to render in a headless environment.

1. Introduction and execution model

TermXT is a lightweight scripting layer over xTerminal commands. A script file is read line by line. Each non-empty, non-comment line is either handled as a language statement such as **set**, **if**, **loop**, or **call**, or it is treated as an xTerminal command and executed through the command repository.

- **Execution is imperative.** The interpreter does not build an AST; it walks the source text with a program counter.
- **Variables are strings.** Numeric behavior exists only when a feature explicitly parses a value as a number, such as math expressions, numeric comparisons, loop counts, or ranges.
- **Functions are pre-scanned.** The engine records each func block before execution so calls can jump into the stored block range.
- **Most state is global.** Variables live in a single dictionary and persist across the whole script. Function arguments temporarily reuse keys like {1} and {2}.

Core runtime ideas

Aspect	Behavior in the implementation
Script source	Read completely with File.ReadAllLines(path)
Execution cursor	Stored in _pc and advanced statement by statement
Stop flags	_stopRequested, _breakRequested, _continueRequested, _returnRequested
Variable store	Dictionary<string, string> with case-insensitive keys

Function registry	Dictionary<string, (Start, End)> built by PreScanFunctions()
Math evaluator	.NET DataTable.Compute after a character whitelist check

2. Invoking the xt command

The outer command supports help, template creation, syntax checking, and execution with optional positional parameters.

```
xt -h
xt -new myscript.xt
xt -check deploy.xt
xt deploy.xt
xt deploy.xt -p production 8080 "Release Candidate"
```

CLI forms

Form	Meaning
xt -h	Print the built-in help message
xt -new <script.xt>	Create a new script template file
xt -check <script.xt>	Validate block structure and selected statement syntax without running
xt <script.xt>	Run a script
xt <script.xt> -p <args>	Run a script and expose arguments as {1}, {2}, ...

Parameter parsing is simple:

- Only double quotes are recognized for grouping arguments.
- Escaped quotes are not implemented in the outer xt command parser.
- If a quoted argument is missing its closing quote, the parser takes the rest of the input as that argument.
- The script path is sanitized through FileSystem.SanitizePath using the current directory read from GlobalVariables.currentDirectory.

3. Script structure, comments, whitespace, and interpolation

Each source line is trimmed before execution. Empty lines and lines beginning with # are ignored.

```
# A comment
set env = production

print "Deploying to {env}"
```

```
time              # because unknown script keywords fall back to xTerminal
commands
```

Lexical rules

Topic	Actual behavior
Comments	A line is a comment only when its trimmed form starts with #.
Inline comments	Not supported as a language feature. Text after a command stays part of the command line.
Keyword matching	Case-insensitive. The first space-delimited token selects the statement.
Interpolation syntax	Only {name} where name matches \w+ (letters, digits, underscore).
Undefined variable	Interpolation leaves the original token unchanged, e.g. {missing} stays {missing}.

Interpolation happens before most statements are interpreted. **DATE**, **TIME**, and **CWD** are dynamic built-ins; **USER** and **PC** are initialized once when the engine is constructed.

> **Note.** Because interpolation occurs early, writing literal braces is only reliable when the name inside the braces is not recognized as an existing built-in or user variable.

4. Variables, assignment, and right-hand-side transforms

Assignment uses **set <var> = <rhs>**. The right-hand side can be plain text or one of the built-in transforms handled by *EvalRhs*.

Right-hand-side forms

Form	Result
`set x = hello`	Stores hello
`set x = "hello world"`	Stores hello world (outer quotes trimmed)
`set x = eval 2 + 3 * 4`	Evaluates a math expression and stores the result as text
`set x = upper hello`	Stores HELLO
`set x = lower HELLO`	Stores hello
`set x = len hello`	Stores 5
`set x = trim "  hi  "`	Stores hi

`set x = substr hello 1 3`	Stores ell
`set x = replace "hello world" "world" "there"`	Stores hello there

```
set env = production
set loud = upper {env}
set chars = len {env}
set sample = substr "hello world" 0 5
set swapped = replace "hello world" "world" "server"
print "{loud} / {chars} / {sample} / {swapped}"
```

Math evaluation details:

- Before evaluation, variables in the expression are interpolated into the line.
- The implementation only allows digits, whitespace, +, -, *, /, %, parentheses, and decimal points through a regular-expression whitelist.
- Evaluation is delegated to DataTable.Compute; the final value is converted to double and then formatted with invariant culture.
- On failure, the engine prints an [xt line N] math error and returns 0 as the expression result.

String transform details:

- upper, lower, len, and trim remove one layer of outer double quotes if present.
- substr takes the last two tokens as start and length, joins everything before them back into the source text, then trims outer quotes from that reconstructed text.
- replace tokenizes with a simple quote-aware parser. Quoting is recommended whenever any argument contains spaces.
- replace uses case-insensitive replacement via StringComparison.OrdinalIgnoreCase.

5. Input, output, command execution, and pipelines

TermXT can print text, execute xTerminal commands, capture command output, and prompt the user for a value.

I/O and command statements

Statement	Behavior
`print "text"`	Writes text after escape processing and interpolation
`run <command>`	Executes an xTerminal command or pipeline
`capture <var> = <command>`	Executes a command and stores captured stdout (or pipe output) in a variable
`input <var> = "prompt"`	Displays a prompt and reads one line from the console
`<plain command>`	Any unrecognized script keyword falls back to

	xTerminal command execution automatically

```
print "Hello {USER}"
run time
capture files = ls
capture exeFiles = ls | cat -s exe
input answer = "Continue"
print "You typed: {answer}"
```

Escapes in print/write/append text:

Escape	Meaning
`\n`	New line
`\t`	Tab
`\\`	Literal backslash

> **Note.** print is implemented from the raw source line rather than the already-interpolated line, so escapes are processed first and interpolation is applied after that.

Pipeline behavior:

- Pipelines are split on single | characters only.
- A double pipe || is intentionally preserved and not treated as a pipeline separator.
- When multiple stages are present, GlobalVariables.isPipeCommand and related counters are set before the stages run and reset afterward.
- If CommandRepository.GetCommand returns null for a stage, the stage is silently skipped by this interpreter.

6. Conditions and branching

Branching uses **if**, optional **elif**, optional **else**, and a closing **end**.

Comparison operators

Operator	Meaning
`==`	Case-insensitive equality
`!=`	Case-insensitive inequality
`>`	Numeric compare when both sides parse as numbers; otherwise string compare
`<`	Numeric compare when possible; otherwise string compare
`>=`	Numeric compare when possible; otherwise string compare

`<=`	Numeric compare when possible; otherwise string compare
`contains`	Case-insensitive substring test
`startswith`	Case-insensitive prefix test
`endswith`	Case-insensitive suffix test

```
if {env} == production
    print "Prod deploy"
elif {env} startswith stage
    print "Staging deploy"
else
    print "Other deploy"
end
```

Logical condition handling:

- not <condition> negates a nested condition.
- || is handled before the routine checks for &&, which effectively gives && higher precedence in mixed expressions because each OR segment is evaluated recursively.
- A condition without an explicit operator is treated as truthy unless it is empty, 0, or false (case-insensitive).
- Comparisons trim one layer of outer double quotes from each side before testing.

> **Note.** The help text says logical operators are evaluated left to right. The actual implementation recursively evaluates not first, then OR splits, then AND splits, then the base comparison.

7. Loops and iteration

The language supports fixed-count loops, while loops, comma-list iteration, numeric ranges, and line-by-line iteration over a captured variable.

Loop forms

Form	Description
`loop <n> ... end`	Repeat the body n times; {i} is the 1-based iteration counter
`while <condition> ... end`	Re-evaluate the condition each time before entering the body
`each <var> in a,b,c ... end`	Iterate over comma-separated values
`each <var> in <start>..<end> ... end`	Iterate an inclusive numeric range; descending ranges are also supported
`each <var> in lines:{name} ... end`	Split variable contents on CR/LF boundaries and iterate over non-empty lines

```
loop 3
    print "Loop #{i}"
end

set countdown = 3
while {countdown} > 0
    print "T-{countdown}"
    set countdown = eval {countdown} - 1
end

each host in server1,server2,server3
    print "Checking {host}"
end

capture files = ls
each f in lines:{files}
    print "{i}: {f}"
end
```

Loop control:

- break exits the current loop.
- continue skips the remainder of the current iteration.
- The checker reports break or continue outside a loop as a syntax error.
- Nested blocks propagate the break/continue flags back to the loop handler, which then clears them appropriately.

8. Functions and return values

Functions are declared with **func <name>** and terminated with **end**. During normal top-level execution, function blocks are skipped. Calls jump into the pre-scanned body range.

```
func greet
    print "Hello, {1}"
    return "ok"
end

call greet "Release Team"
print "Result: {result}"
```

Function mechanics

Mechanic	Implementation behavior
Arguments	call tokenizes its argument list with the quote-aware parser and stores them as {1}, {2}, ...
Scope	Most variables are global. Only positional argument keys are temporarily overridden and later restored.
Return value	return <value> stores into {result} and sets a return flag
Missing return	{result} keeps its previous value if a function

	finishes without return
Recursion	Possible in principle because functions call back into ExecuteBlock

> **Note.** return is only valid inside a function according to the checker. The checker does not enforce function signatures because functions are name-only and accept a variable number of positional arguments.

9. Error handling and status variables

The language exposes two related mechanisms: status variables that reflect the last command outcome, and structured **try** / **catch** / **end** blocks.

```
try
    run somecommand
    read text = missing.txt
catch
    print "Caught: {error_message}"
end

if {error} == true
    print "Something failed"
end
```

- **error** is initialized to false and updated by run, capture, read, and try.
- **error_message** is initialized to an empty string and populated by try when an exception is caught or when read reports a missing file.

Important try/catch details

Detail	Actual behavior
Catch trigger	catch runs not only for thrown exceptions, but also when GlobalVariables.isErrorCommand becomes true inside the try block
Exception source	Interpreter file methods can throw; xTerminal commands may set error flags rather than throw exceptions
After catch	If the catch block completes successfully, the global error flag may be restored to the saved incoming value
Checker rule	catch is valid only when paired with a try block

10. File operations and timing

Scripts can read a whole file into a variable, overwrite a file, append to a file, pause execution, or stop the script entirely.

Operational statements

Statement	Behavior
`read <var> = <file>`	Reads the full file contents into <var>; on failure sets error and error_message
`write <file> "text"`	Writes text to the file and appends Environment.NewLine
`append <file> "text"`	Appends text plus Environment.NewLine
`wait <ms>`	Sleeps for the specified positive integer millisecond count
`exit`	Stops script execution immediately

```
write output.txt "Started at {TIME}"
append output.txt "User: {USER}"
read contents = output.txt
print "{contents}"
wait 500
exit
```

Note. write and append always add a trailing newline. The file path is sanitized relative to the current directory after interpolation.

11. Built-in variables and runtime state

Built-in variables

Variable	Meaning
`{DATE}`	Current date formatted as yyyy-MM-dd; resolved dynamically on each interpolation
`{TIME}`	Current time formatted as HH:mm:ss; resolved dynamically on each interpolation
`{USER}`	Current account name from GlobalVariables.accountName; initialized once per engine
`{PC}`	Computer name from GlobalVariables.computerName; initialized once per engine
`{CWD}`	Current working directory read from GlobalVariables.currentDirectory at interpolation time
`{i}`	Current loop iteration counter (1-based)
`{result}`	Return value from the last function call that executed

	return
`{error}`	String true or false
`{error_message}`	Latest stored interpreter/catch error message
`{1}, {2}, ...`	Script parameters or function call positional arguments

Case sensitivity:

- The variable dictionary uses StringComparer.OrdinalIgnoreCase.
- Interpolation keys are therefore case-insensitive for stored variables and built-ins like DATE, TIME, and CWD.
- String comparisons such as ==, contains, startswith, and endswith are also case-insensitive.

12. Script checker behavior

Before a script runs, the outer xt command performs an automatic check. The check can also be called directly with **xt -check <file>**.

What the checker validates

Validated rule	Examples
`Block matching`	if/loop/each/func/try/while must be closed with end
`elif / else placement`	Must occur inside an if block
`catch placement`	Must occur inside a try block
`Assignment presence`	set, capture, read, and input must contain =
`break / continue placement`	Must appear inside loop, each, or while
`return placement`	Must appear inside a func block

What the checker does not validate:

- It does not parse math syntax deeply.
- It does not verify that called functions exist.
- It does not verify that xTerminal commands exist.
- It does not inspect file paths, interpolation, or operator spelling beyond the broad structure of a line.

13. Cookbook examples

13.1 Deployment example

```
set env = {1}
set port = {2}
print "Deploying to {env} on port {port}"
```

```
run time

each host in server1,server2,server3
    print "Checking {host}"
    run ping {host}
end

set total = eval 10 + {port} * 2
print "Computed: {total}"

set upper_env = upper {env}
set envLen = len {env}
print "{upper_env} has {envLen} characters"

set countdown = 3
while {countdown} > 0
    print "T-{countdown}"
    set countdown = eval {countdown} - 1
end

print "Done!"
```

13.2 Iterate over captured output

```
capture files = ls
each f in lines:{files}
    if {f} endswith .exe
        print "Executable: {f}"
    end
end
```

13.3 Reusable function with return value

```
func classify
    if {1} >= 90
        return A
    elif {1} >= 80
        return B
    else
        return C
    end
end

call classify 87
print "Grade: {result}"
```

13.4 Simple log file writer

```
write deploy.log "Start {DATE} {TIME}"
append deploy.log "Machine={PC}"
append deploy.log "User={USER}"
```

14. Implementation quirks and caveats

- **Unknown keywords fall through to xTerminal execution.** That means a line like time or ls can be written without run.
- **Inline comments are not stripped.** A # character in the middle of a command line is ordinary text to the interpreter.
- **Comma iteration is not quote-aware.** each x in a,"b,c",d will still split on every comma.
- **result is sticky.** A function that does not execute return leaves the prior {result} value intact.
- **write/append add a newline automatically.** This is true even when the provided text already ends with one.
- **Function locals do not exist.** Ordinary variables set inside a function stay globally visible after the call.
- **The checker is intentionally shallow.** A script can pass -check and still fail at runtime.
- **The language is implemented for Windows.** The command class is decorated with [SupportedOSPlatform("Windows")].

Help text vs. implementation highlights

Topic	Documentation/help text	Observed implementation detail
Logical evaluation	Says && and || are evaluated left to right	The condition routine checks not, then OR splits, then AND splits recursively
Pipes	Help says commands support pipes	True; single | splits stages and || is preserved
each numeric range	Examples show ascending ranges	Descending inclusive ranges are also supported
Unknown commands	Not explicitly described	Unrecognized script keywords are treated as xTerminal commands

15. Quick reference

Statement summary

Statement	Short form / reminder
set	set name = value | eval expr | upper/lower/len/trim/substr/replace
print	print "text with {vars}"
run	run any xTerminal command
capture	capture out = somecommand | anothercommand
input	input name = "Prompt"
if / elif / else / end	Branching

`loop / end`	Repeat fixed number of times
`while / end`	Repeat while condition is truthy
`each / end`	Comma list, numeric range, or lines:{var}
`func / call / return`	Reusable block and return value in {result}
`try / catch / end`	Structured failure handling
`read / write / append`	File I/O
`wait`	Sleep in milliseconds
`break / continue`	Loop control
`exit`	Stop script

End of manual